Geometric Spanning Trees and Hypergraphs that Minimizes the Wiener Index

Atishaya Maharjan

University of Manitoba
Geometric, Approximation, and Distributed Algorithms (GADA) lab

July 7, 2025

This week

This week, I aimed to do the following:

- Rigorously analyze and obtain the worst case of the divide and conquer algorithm for approximating Hamiltonian paths.

This week

This week, I aimed to do the following:

- Rigorously analyze and obtain the worst case of the divide and conquer algorithm for approximating Hamiltonian paths.
- Refactor and decouple the algorithm that generates the tests and publish it on GitHub.

This week

This week, I aimed to do the following:

- Rigorously analyze and obtain the worst case of the divide and conquer algorithm for approximating Hamiltonian paths.
- Refactor and decouple the algorithm that generates the tests and publish it on GitHub.
- Think about a simplified version of the Happy Edge Conjecture (NCST) [More on this later].

Divide and Conquer Algorithm for Approximating Hamiltonian Paths

Algorithm 1 DivideAndConquerWiener(P)

procedure DivideAndConquerWiener(P, $depth = 0$, $maxDepth = 10$)
  if $|P| \leq 4$ or $depth > maxDepth$ **then**
    return BruteForceHamiltonianPath(P)
  end if
  $(a, b, c) \leftarrow$ FindBisectingLine(P)
  $(P_L, P_R) \leftarrow$ PartitionPoints(P, a, b, c)
  $\pi_L \leftarrow$ DivideAndConquerWiener(P_L, $depth + 1$)
  $\pi_R \leftarrow$ DivideAndConquerWiener(P_R, $depth + 1$)
  return ConnectPaths(π_L, π_R)
end procedure

Supporting Procedure: FindBisectingLine

Algorithm 2 FindBisectingLine(P)

procedure FindBisectingLine(P)
  $minX \leftarrow \min_{p \in P} p.x$, $maxX \leftarrow \max_{p \in P} p.x$
  $minY \leftarrow \min_{p \in P} p.y$, $maxY \leftarrow \max_{p \in P} p.y$
  $width \leftarrow maxX - minX$, $height \leftarrow maxY - minY$
  if $width \geq height$ **then**
    $midX \leftarrow (minX + maxX)/2$
    return $(1.0, 0.0, -midX)$ ▷ Vertical line: $x = midX$
  else
    $midY \leftarrow (minY + maxY)/2$
    return $(0.0, 1.0, -midY)$ ▷ Horizontal line: $y = midY$
  end if
end procedure

Supporting Procedure: PartitionPoints

Algorithm 3 PartitionPoints(P, a, b, c)

```
1:  procedure PartitionPoints(P, a, b, c)
2:      L ← [ ], R ← [ ]
3:      for each p ∈ P do
4:          v ← ap.x + bp.y + c
5:          if v ≤ 0 then
6:              L ← L ∪ {p}
7:          else
8:              R ← R ∪ {p}
9:          end if
10:     end for
11:     if L = ∅ then
12:         Move one point from R to L
13:     else if R = ∅ then
14:         Move one point from L to R
15:     end if
```

Supporting Procedure: ConnectPaths

Algorithm 4 ConnectPaths(π_1, π_2)

procedure ConnectPaths(π_1, π_2)
    options $\leftarrow$ empty list
    Append $(\pi_1 + \pi_2, W(\pi_1 + \pi_2))$ to *options*
    Append $(\pi_1 + \text{reverse}(\pi_2), W(\pi_1 + \text{reverse}(\pi_2)))$ to *options*
    Append $(\text{reverse}(\pi_1) + \pi_2, W(\text{reverse}(\pi_1) + \pi_2))$ to *options*
    Append $(\text{reverse}(\pi_1) + \text{reverse}(\pi_2), W(\text{reverse}(\pi_1) + \text{reverse}(\pi_2)))$ to *options*
    return path with minimum Wiener index from *options*
end procedure

Worst case?? Approximation Ratio???

- When experimenting with different configurations, I realized that drawing the configuration trees helped.

Worst case?? Approximation Ratio???

- When experimenting with different configurations, I realized that drawing the configuration trees helped.
- Also, it always seemed that the algorithm was struggling for certain configuration trees. Specifically, it struggled at the merge step of reconciling a node with each sibling node if two close points did not share the same parent.

Worst case?? Approximation Ratio???

- When experimenting with different configurations, I realized that drawing the configuration trees helped.
- Also, it always seemed that the algorithm was struggling for certain configuration trees. Specifically, it struggled at the merge step of reconciling a node with each sibling node if two close points did not share the same parent.

Chalk & Talk

How do we fix this?

- The problem is due to the fact that there may be cases where the algorithm merges two points that are not close to each other, which results in a longer path or having to go through the same edges.

How do we fix this?

- The problem is due to the fact that there may be cases where the algorithm merges two points that are not close to each other, which results in a longer path or having to go through the same edges.

- We cannot fix this by simply adding a condition to check if the points are close to each other, as this would violate the divide and conquer principle of no overlapping subproblems.

Proposed fix!

Revamp: Need to reconsider the merge step. How?

For each edge in π_R and π_L at the ith step:

1. Pick the closest edge between the two paths π_L and π_R. (How?)

Proposed fix!

Revamp: Need to reconsider the merge step. How?

For each edge in π_R and π_L at the ith step:

1. Pick the closest edge between the two paths π_L and π_R. (How?)
2. For each edge in π_L and π_R, check if moving the edge to join the head and tail of the path then merging the two paths results in a shorter Wiener index.

Proposed fix!

Revamp: Need to reconsider the merge step. How?

For each edge in π_R and π_L at the ith step:

1. Pick the closest edge between the two paths π_L and π_R. (How?)
2. For each edge in π_L and π_R, check if moving the edge to join the head and tail of the path then merging the two paths results in a shorter Wiener index.
3. Pick the smallest Wiener index from the options.

Proposed fix!

Revamp: Need to reconsider the merge step. How?

For each edge in π_R and π_L at the ith step:

1. Pick the closest edge between the two paths π_L and π_R. (How?)
2. For each edge in π_L and π_R, check if moving the edge to join the head and tail of the path then merging the two paths results in a shorter Wiener index.
3. Pick the smallest Wiener index from the options.

In total, we have $O(n^2)$ options to check, where n is the number of points in the configuration. So this bumps up the time complexity of the Divide and Conquer algorithm to $O(n^2 \log n)$.

Proposed fix!

Revamp: Need to reconsider the merge step. How?

For each edge in π_R and π_L at the ith step:

1. Pick the closest edge between the two paths π_L and π_R. (How?)
2. For each edge in π_L and π_R, check if moving the edge to join the head and tail of the path then merging the two paths results in a shorter Wiener index.
3. Pick the smallest Wiener index from the options.

In total, we have $O(n^2)$ options to check, where n is the number of points in the configuration. So this bumps up the time complexity of the Divide and Conquer algorithm to $O(n^2 \log n)$.
I haven't fully analyzed the approximation ratio if we use this approach, but intuitively, it should be better.

Interlude: NCSTs & The Happy Edge Conjecture

The Happy Edge Conjecture

Given two trees T_I and T_F embedded on the same vertex set and let $\mathcal{O}$ denote the optimal set of reconfiguration sequences from T_I to T_F. Then, for each edge $e \in T_I \cap T_F$, we have that $e \notin \mathcal{O}$.

Interlude: NCSTs & The Happy Edge Conjecture

The Happy Edge Conjecture

Given two trees T_I and T_F embedded on the same vertex set and let $\mathcal{O}$ denote the optimal set of reconfiguration sequences from T_I to T_F. Then, for each edge $e \in T_I \cap T_F$, we have that $e \notin \mathcal{O}$.

- We know that the Happy Edge Conjecture is true for trees with at all of their happy edges being boundary edges.

Interlude: NCSTs & The Happy Edge Conjecture

The Happy Edge Conjecture

Given two trees T_I and T_F embedded on the same vertex set and let $\mathcal{O}$ denote the optimal set of reconfiguration sequences from T_I to T_F. Then, for each edge $e \in T_I \cap T_F$, we have that $e \notin \mathcal{O}$.

- We know that the Happy Edge Conjecture is true for trees with at all of their happy edges being boundary edges.
- Connor and I met up with Dr. Zhu on Thursday to see where GADA lab's proof of the Happy Edge Conjecture fails.

Interlude: NCSTs & The Happy Edge Conjecture

The Happy Edge Conjecture

Given two trees T_I and T_F embedded on the same vertex set and let $\mathcal{O}$ denote the optimal set of reconfiguration sequences from T_I to T_F. Then, for each edge $e \in T_I \cap T_F$, we have that $e \notin \mathcal{O}$.

- We know that the Happy Edge Conjecture is true for trees with at all of their happy edges being boundary edges.
- Connor and I met up with Dr. Zhu on Thursday to see where GADA lab's proof of the Happy Edge Conjecture fails.
- The GADA lab's proof is of the following flavour:
  1. Base case: Happy Edge Conjecture is true for edges in the convex hull (Boundary edges).
  2. Inductive step: Pick an interior happy edge and split the point set into two non-intersecting convex sets. Then, the interior edge is now a boundary edge.
  3. Rinse and Repeat.

The Happy Edge Conjecture (Contd.)

The problem crops up in the merging step between the distinct convex sets. The GADA lab's proof assumes that the two convex sets are disjoint, but this is unproven. In general, it is sufficient to prove the following:

Simplified Happy Edge Conjecture

Suppose every optimal flip sequence $\mathcal{O}$ from T_I to T_F flips at least one happy edge. Now, consider all $\sigma \subseteq \mathcal{F} \in \mathcal{O}$ that begins by flipping a happy edge e. Then, there exists a σ' such that $|\sigma'| < \sigma$ where $e \notin \sigma'$.

Simplified case of the Happy Edge Conjecture (Contd.)

To start, we begin by analyzing the case where the removing the happy edge e creates two disjoint components C_1 and C_2 such that WLOG $|C_1| = 1$.

Chalk & Talk

Next week tasks!

- Implement the proposed fix to the divide and conquer algorithm and analyze the approximation ratio.
- Continue working on the Happy Edge Conjecture and try to prove the simplified version.
- Think of a way to bound the base case (Connecting two points) of the divide and conquer algorithm to the actual optimal solution. Currently, the analysis does not make any sense.

The end.

maharjaa@umanitoba.ca